Data Quality Workbench

1. What was built and why

A command-line Python tool that turns a structured but inconsistent contact export into a clean file and a traceable review queue. A person preparing a CRM migration needs to know both which rows are usable and why other rows were excluded. A cleaned file alone cannot explain whether an important record was silently lost or an ambiguous record was merged incorrectly.

The intended outcome is a reviewable import candidate, not an automatic change to a live CRM. The implementation provides deterministic rules, explicit dispositions for every source row, and reproducible examples. It does not use a trained model or an LLM to decide which records are correct.

2. Input and output contract

Input is one UTF-8 or UTF-8-with-BOM CSV containing exactly six columns. Column order may vary; missing, repeated or additional columns are rejected. Each row must supply string values for all six fields.

Field	Meaning and demo rule
record_id	Contact key, normalized to C followed by five digits
name	Required; trim outer space and collapse internal whitespace
email	Required; lowercase and apply a basic syntax check
country	Explicit aliases for US, GB, CN and AE only
joined_on	A valid calendar date written as YYYY-MM-DD
plan	One of basic, pro or enterprise after normalization

The CLI writes cleaned.csv, row_ledger.csv, evidence.json and report.html into a new directory. JSON retains each original row, normalized values, changed-field names, source line and disposition. The HTML report filters rows by outcome. sample_review.xlsx is a separate prebuilt review snapshot; XLSX generation is not part of this Python CLI.

3. How the decision process works

1. Apply Unicode NFKC normalization and whitespace cleanup; standardize ID, email, country and plan using the documented policy.

2. Validate field values. An invalid ID cannot be used for grouping; that row is quarantined.

3. Group remaining rows by normalized ID. If any member is invalid, hold the entire group for review.

4. If a valid group contains different normalized records, quarantine all members. Do not select a winner by recency or guess which values are true.

5. If all normalized values agree, retain the first source row and classify later rows as duplicates. Distinct IDs are never fuzzy-merged.

6. Export results and assert that all input rows have one outcome and all accepted IDs are unique.

4. Data provenance and worked examples

demo.py creates the input deterministically in Python; no external dataset, client export or personal contact information is used. Names use "Demo Contact" and addresses use example.test. The fixture consists of 1,000 base rows, 100 copies of rows 101-200, 20 added conflicting variants of the first 20 IDs, and 60 invalid rows: 20 missing IDs, 20 malformed emails and 20 unknown countries. Dates and subscription plans are generated by fixed arithmetic rules, with no random seed required.

For example, " c00101 " becomes C00101, " CONTACT0101@EXAMPLE.TEST " becomes contact0101@example.test, and "United Kingdom" becomes GB. Its copied row has the same normalized values, so one is accepted and one is marked duplicate. In contrast, the two rows with ID C00001 specify different plans. Both are held for review; neither is silently preferred. These examples describe actual fixture construction, not customer records.

5. Acceptance criteria and observed results

Check	Expected for this fixture	Observed
Input row count	1,180	1,180
Accepted records	1,000 base IDs minus 20 conflicting IDs	980
Redundant source rows	100 exact normalized copies	100
Review queue	40 conflict-group rows plus 60 invalid rows	100
Reconciliation	980 + 100 + 100 = 1,180	Zero unaccounted rows
Repeat processing	Accepted rows remain unchanged	Passed automated check

Seventeen test methods passed under Python 3.12.14 on macOS. They cover the fixture totals, normalization, exact duplicates, group-wide conflict/invalid handling, dates, unknown fields, input preservation, stable reprocessing, formula-prefix neutralization, HTML escaping, invalid headers and no-overwrite behavior. Browser review confirmed that the HTML "Needs review" filter displays 100 rows.

The Excel snapshot's counts were recalculated in the authoring engine. Changing one ledger status from accepted to quarantined changed counts from 980/100 to 979/101; the original status was restored. Native Microsoft Excel was not exercised.

These are conformance checks against a deliberately constructed fixture, not a measured accuracy rate on a representative real-world population. There was no independent annotation study, holdout dataset, throughput benchmark, manual-labor comparison or measured business ROI.

6. Reproduce and inspect

Python 3.10+ is the declared minimum; the recorded run used Python 3.12.14. The CLI and tests use the standard library only. Run from the repository directory and choose output paths that do not exist:

```
python -m unittest -v
python demo.py --output demo-output
python data_quality.py sample_input.csv --output another-run
```

Open demo-output/results/report.html locally. Compare cleaned.csv against the accepted records in evidence.json, and inspect any excluded source line in the ledger. The JSON includes the input SHA-256 so a reviewer can identify the exact input bytes. A hash establishes identity, not the truth of the source values.

Evidence	Location
Rules, grouping and output code	data_quality.py
Full synthetic fixture generator	demo.py
Executable acceptance checks	test_data_quality.py
Downloadable input and cleaned example	sample_input.csv; sample_cleaned.csv
Review interfaces	sample_report.html; sample_review.xlsx
Recorded test run and source hashes	verification.json

7. Limits and client adaptation

The country map, ID format, allowed plans and email-lowercasing rule are demo policies, not universal facts about contact data. Email syntax does not establish mailbox ownership or deliverability. No fuzzy entity resolution, missing-value imputation, external enrichment or live CRM write is implemented. If two genuinely different contacts share one ID, a reviewer must resolve the conflict upstream.

Human-facing CSV exports prefix formula-like cells; JSON preserves the exact originals. The target spreadsheet or import application must still be tested because interpretation varies. Processing reads the dataset into memory; no production-scale memory or speed claim is made. Output validation reduces accidental loss but is not an all-or-nothing filesystem transaction if a disk write fails.

For a client engagement, agree on the real schema and cleaning policy first. Obtain an approved, preferably de-identified sample; identify ambiguous records that require a client decision; run the pipeline in a separate directory; reconcile counts and spot-check transformations; then have the client approve an import candidate. A live import is a separate authorized milestone with backup and rollback requirements.

8. Contribution and evidence boundary

This is an independent portfolio project built with AI-assisted implementation, testing and documentation. It is not paid client work and contains no claimed employment, customer outcome or deployed CRM integration. The code, test fixture and stated checks are the evidence for the demonstrated engineering capability.